SKT SOFTWARE SOLUTION	**EMPRESA DE DESARROLLO E INNOVACIÓN**	**CÓDIGO:** MI-DES-ESP-003
	PROCESO: DESARROLLO DE SOFTWARE	**VERSIÓN:** 1.0
	Desarrollo del Sistema LUPSI	**ELABORACIÓN:** 21/04/2026
	Especificación del Backend: Motor de Reservas Médicas	**ÚLTIMA REVISIÓN:** 21/04/2026

1. Título

Especificación Técnica del Motor de Reservas Médicas: Gestión de Horarios, Disponibilidad de Franjas y Persistencia de Citas en el Ecosistema LUPSI (Sprint 3)

2. Introducción

El módulo de reservas constituye el núcleo operativo del sistema LUPSI, siendo el componente que materializa la propuesta de valor principal de la plataforma: permitir que un paciente agende una cita con un profesional médico de manera autónoma, sin fricciones y con garantías de integridad de datos.

Este documento describe la implementación técnica del *Motor de Reservas* en el backend NestJS, incluyendo: la generación dinámica de franjas horarias disponibles, la prevención de colisiones de horario (*double-booking*), la gestión de zonas horarias y los mecanismos de seguridad que aseguran que únicamente los pacientes autenticados puedan persistir una cita en la base de datos.

3. Objetivo

Documentar formalmente los algoritmos, flujos de datos y decisiones de diseño que conforman el motor de agendamiento médico del sistema LUPSI, garantizando su comprensión, mantenibilidad y auditabilidad por parte del equipo de desarrollo y los revisores del proyecto.

4. Tecnología y Arquitectura

- **Backend Framework**: NestJS (Node.js + TypeScript), arquitectura en módulos desacoplados.
- **Base de Datos**: PostgreSQL alojado en Supabase Cloud con extensión `btree_gist` para restricciones de exclusión de rangos de tiempo.
- **Autenticación**: Supabase Auth con validación directa de sesión en cada petición protegida.

- **Seguridad**: Guard global `JwtAuthGuard` con decorador `@Public()` para rutas de consulta abiertas.
- **Frontend**: Angular 17+ con `AuthInterceptor` que inyecta el `Bearer Token` en cada petición HTTP.

5. Estructura del Módulo

El módulo `AppointmentsModule` agrupa los siguientes componentes:

Archivo	Responsabilidad
`appointments.controller.ts`	Exposición de los endpoints HTTP. Recibe peticiones, delega lógica al servicio y retorna respuestas.
`appointments.service.ts`	Lógica de negocio: generación de slots, verificación de disponibilidad y persistencia de citas.
`appointments.module.ts`	Configuración de dependencias e inyección del módulo Supabase.
`dto/create-appointment.dto.ts`	Objeto de Transferencia de Datos con validación de entrada para crear citas.

Componentes del módulo de citas (Sprint 3).

6. Endpoints Implementados

Método	Ruta	Acceso	Descripción
GET	`/api/v1/appointments/availabl`	Público	Devuelve las franjas horarias libres para un doctor en una fecha concreta.
POST	`/api/v1/appointments`	Privado	Crea y persiste una nueva cita médica para el paciente autenticado.
GET	`/api/v1/appointments`	Privado	Retorna las citas filtradas por fecha y rol del usuario (PATIENT, DOCTOR, RECEPTIONIST).

Los endpoints privados requieren `Bearer Token` en la cabecera HTTP.

7. Motor de Franjas Horarias (Available Slots)

7.1. Descripción del Algoritmo

El método `getAvailableSlots(doctorId, date)` implementa un algoritmo de generación dinámica de horarios disponibles en tres fases:

1. **Consulta de citas ocupadas**: Se extraen de la base de datos todas las citas con estado `SCHEDULED` del doctor solicitado para la fecha indicada, dentro del rango horario completo del día en UTC.

2. **Generación del espacio de franjas**: Se itera sobre el horario laboral configurado (**08:00 a 17:00, horario de Ecuador, UTC-5**), generando bloques de **30 minutos** consecutivos.

3. **Filtro de disponibilidad**: Cada franja generada se compara contra las citas existentes mediante lógica de superposición de intervalos: si `inicio_franja < fin_cita AND fin_franja > inicio_cita`, el bloque se marca como *ocupado* y se excluye del resultado.

7.2. Gestión de Zona Horaria (UTC-5 Ecuador)

Los servidores operan en UTC, mientras que los usuarios trabajan en horario de Ecuador (ECT, `UTC-5`). Una mala conversión generaba franjas horarias incorrectas (e.g., mostrar disponibilidad desde las 03:00 a.m. en lugar de las 08:00 a.m.).

Solución implementada: La construcción de objetos `Date` se realiza usando el formato ISO 8601 con el offset explícito de Ecuador, de modo que el servidor interprete correctamente la hora local sin ambigüedades:

```
// Crear el objeto Date representando la hora exacta en
   ECUADOR (-05:00)
const currentSlotStart = new Date(`${date}T${hh}:${mm
   }:00-05:00`);
const currentSlotEnd   = new Date(currentSlotStart.getTime() +
    30 * 60000);
```

Listing 1: Generación de franja con offset correcto — `appointments.service.ts`

De esta forma, la franja de las 08:00 ECT es almacenada como `13:00Z` en la base de datos (UTC), y el navegador del paciente la muestra correctamente como **08:00 a.m.** gracias a la conversión automática del objeto `Date` de JavaScript.

8. Prevención de Double-Booking

El sistema implementa dos capas de protección contra el agendamiento duplicado de citas:

8.1. Capa 1 — Lógica de Negocio (Servicio)

El método `getAvailableSlots` no retorna las franjas ya ocupadas, impidiendo que el usuario pueda seleccionar un horario tomado desde la interfaz.

8.2. Capa 2 — Restricción de Base de Datos (PostgreSQL EXCLUDE)

La tabla `appointments` posee una restricción de exclusión de nivel de base de datos implementada con la extensión `btree_gist`. Si dos citas del mismo doctor se superponen en el tiempo, la base de datos rechaza la segunda inserción:

```
CONSTRAINT no_overlapping_appointments EXCLUDE USING gist (
    doctor_id WITH =,
    tstzrange(appointment_time, appointment_end_time) WITH &&
) WHERE (status = 'SCHEDULED')
```

Listing 2: Restricción EXCLUDE en el esquema de la base de datos

Cuando se produce una colisión, PostgreSQL retorna el código de error `23P01` (exclusion_violation). El servicio captura este error y responde al cliente con un HTTP `409 Conflict` y un mensaje descriptivo:

```
if (error.code === '23P01') {
  throw new ConflictException(
    'El horario seleccionado ya se encuentra ocupado por otra
       cita.'
  );
}
```

Listing 3: Manejo del 409 en `appointments.service.ts`

9. Flujo de Creación de Cita (Extremo a Extremo)

Paso	Actor	Acción
1	Paciente (Frontend)	Selecciona doctor, fecha y franja horaria disponible en la interfaz.
2	AuthInterceptor	Inyecta automáticamente el `Bearer Token` de la sesión en la cabecera HTTP de la petición.
3	JwtAuthGuard	Valida el token consultando Supabase Auth (`getUser`). Obtiene el rol del usuario desde la tabla `profiles`.
4	AppointmentsController	Recibe el `POST /appointments`. Extrae el `patientId` del objeto `request.user` inyectado por el guard.
5	AppointmentsService	Calcula `appointment_end_time` sumando 30 minutos al inicio. Ejecuta el `INSERT` en la tabla `appointments`.
6	PostgreSQL	Valida la restricción `EXCLUDE`. Si hay colisión retorna `23P01`; de lo contrario confirma la inserción.
7	Paciente (Frontend)	Recibe confirmación de la cita agendada o mensaje de error descriptivo.

Flujo completo de creación de cita en el sistema LUPSI.

10. Sistema de Autenticación y Roles (RBAC)

10.1. Guard Global (JwtAuthGuard)

El `JwtAuthGuard` protege todos los endpoints del sistema por defecto. En cada petición autenticada realiza:

1. Extrae el `Bearer Token` de la cabecera `Authorization`.
2. Llama a `supabase.auth.getUser(token)` para validar la sesión activa directamente con Supabase.

3. Consulta la tabla `public.profiles` para obtener el **rol real** del usuario (`PATIENT`, `DOCTOR`, `RECEPTIONIST`, `ADMIN`).

4. Inyecta el objeto `{id, email, role}` en la solicitud HTTP para su uso en el controlador y servicio.

10.2. Ruta Pública (available-slots)

El endpoint `GET /available-slots` está marcado con el decorador `@Public()`, permitiendo que sea accedido sin sesión activa. Esto es necesario para que el formulario de reserva pueda mostrar los horarios disponibles antes de que el paciente confirme la cita.

10.3. Filtrado por Rol en Agenda Diaria

El endpoint `GET /appointments` devuelve información filtrada según el rol del solicitante:

Rol	Datos devueltos
`PATIENT`	Solo las citas donde `patient_id` coincide con el ID del paciente autenticado.
`DOCTOR`	Solo las citas donde `doctor_id` coincide con el ID del doctor autenticado.
`RECEPTIONIST / ADMIN`	Todas las citas del sistema (vista completa de la agenda diaria).

Control de acceso basado en roles (RBAC) aplicado en el servicio de citas.

11. Estrategia Anti-RLS: Merge en Memoria

La base de datos utiliza el mecanismo de *Row Level Security* (RLS) de Supabase. Las políticas activas generaban recursión infinita al intentar usar `JOIN` entre `appointments`, `profiles` y `patients` a través de la capa HTTP de PostgREST.

Solución adoptada: En lugar de usar `JOIN` en la consulta SQL, el `AppointmentsService` realiza tres consultas independientes y combina los datos en memoria (TypeScript):

1. Consulta principal a `appointments` para obtener la lista de citas.

2. Consulta a `patients` con los IDs de pacientes extraídos del paso 1.

3. Consulta a `profiles` con los IDs combinados de pacientes y doctores.

Esta decisión de diseño garantiza la estabilidad del sistema ante las limitaciones de RLS, manteniendo un número fijo y predecible de tres consultas por solicitud, lo cual resulta aceptable para el volumen de datos del sistema médico.

12. Responsabilidades

Responsable	Rol en el Módulo
Daniel Luisa	Implementación del `AppointmentsService`, lógica de generación de slots, algoritmo de superposición de intervalos y gestión de errores.
Sebastián Ortiz	Diseño del esquema SQL (restricción EXCLUDE), corrección de zona horaria (UTC-5 Ecuador) y módulo de autenticación (`JwtAuthGuard` + RBAC).

13. Firmas de Responsabilidad

Rol	Nombre / Representante	Firma (QR)	Fecha
Gestor del Proyecto	Angel Ayuquina		20/04/2026
Arquitecto BD y Cloud	Sebastián Ortiz		20/04/2026
Desarrollador Backend y QA	Daniel Luisa		20/04/2026
Desarrollador Frontend y UX/UI	Alex Huachi		20/04/2026

14. Control de Cambios

Versión	Fecha	Descripción del Cambio	Responsable
1.0	20/04/2026	Creación del documento. Motor de reservas, slots, double-booking, RBAC y zona horaria.	Daniel Luisa / Sebastián Ortiz